UNIPROCESSOR SCHEDULING

In this chapter we will take a closer look at the scheduling decisions that the operating system has to make in a multiprogramming environment. As indicated earlier, each process is in a certain state: running, ready, blocked, suspended-ready, etc. The scheduling mechanism will to a large extent determine how the state of a certain process evolves. Often, these decisions are divided into three sub-problems: *long-term*, *medium-term* and *short-term* scheduling. The emphasis of this chapter lies on the third form of scheduling.

Long-term and medium-term scheduling

The influence of long-term scheduling is mainly situated in the determination of the number of active processes. When a new process arrives, it will remain inactive until the long-term scheduler decides to add it to the list of active processes. When activation occurs, the state of the process changes to ready or *ready-suspend*.

All new processes together form a queue and are served by the long-term scheduler. This scheduler will have to make two types of decisions:

1. When can a new process be activated?
2. Which process is best to activate out of all the new, waiting processes?

The desired degree of multiprogramming plays a major role in the first case. More active processes also gives rise to more processes that are ready, which improves processor efficiency. On the other hand, more active processes increases the execution time of the active processes (and requires more virtual memory). The long-term scheduler will closely monitor the idle time of the processor and activate processes when the idle time becomes too high.

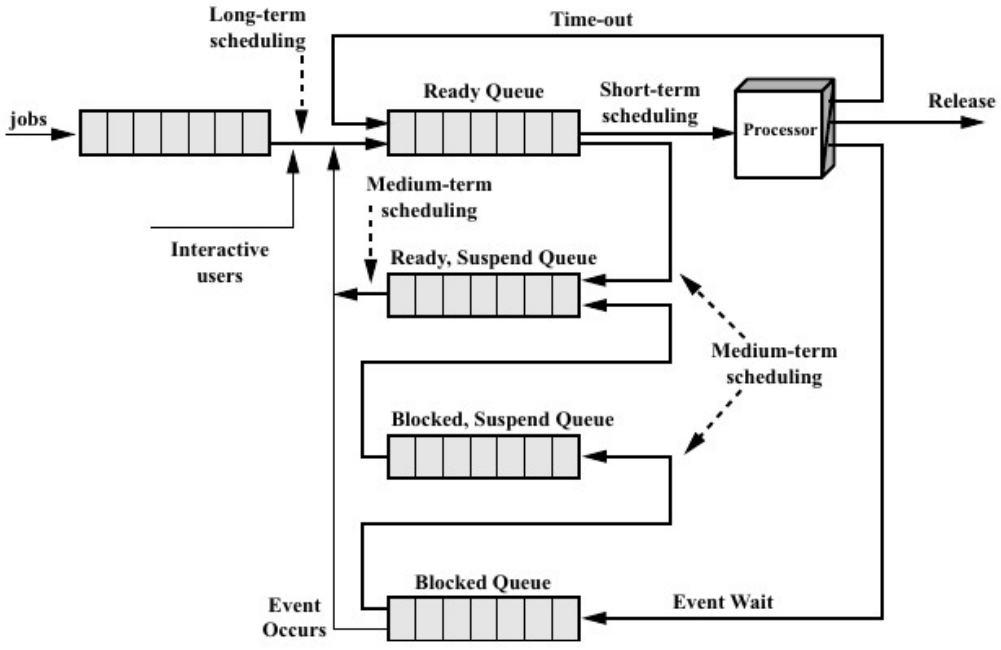

Fig. 37: Influence of scheduling on process state [Stallings 03].

For the choice of the process to be activated, the nature of the process can also play a decisive role. Processes are often divided into *processor-bound* and *I/O-bound* processes. The latter set are the ones that frequently use I/O operations, while processor-bound processes require a lot of computation and have less need for I/O operations. Too many I/O-bound processes can lead to a situation where all processes are blocked. Selecting only processor-bound processes leads to an inefficient use of I/O devices.

Furthermore, interactive processes are often allowed to join the list of active processes immediately, until a certain threshold is reached. If there are any more interactive processes, they will be aborted and the user will be informed.
Medium-term scheduling occurs slightly more frequently and determines which processes are (partially) present in physical memory. This form of scheduling is therefore closely related to virtual memory management, or more precisely, to resident set management.

1 Non-real-time short-term scheduling

The short-term scheduling function indicates which of the ready processes may use the processor, or is running. The functionality of different forms of scheduling is also shown in Figure 37. Before we can compare various scheduling algorithms, it is important to consider how we determine whether or not such an algorithm performs well.

1.1 Performance measures

A first criterion is the (average) *turnaround time* of a process. This indicates how long it takes before a process is completely executed, starting from the moment it was started. This time also includes the time that a process is blocked or has to wait in the ready state. The *response time* is strongly related to this and indicates the time until a user receives feedback that his process is running. This last measure is mainly important from the point of view of the users in interactive systems and to a lesser extent for the system itself.

In addition to a low response and turnaround time, it is also desirable that the processor is almost always working, so that no capacity is lost. This is referred to as *processor utilization*. A high utilisation often results in a larger turnaround time (or waiting time). A single algorithm can therefore never optimise all performance measures simultaneously. *Fairness* and *predictability* are also of some value. The former indicates the extent to which the capacity of the resources is fairly distributed over the various processes. The definition of what is fair has been the subject of many discussions in the past and will not change in the future. A commonly used value with respect to fairness is the normalized turnaround time. This is the turnaround time of a process divided by the time that the process effectively had access to the processor. Finally, predictability indicates how sensitive the execution time of a process is to the presence of other processes. A large variation in the execution time is not desirable.

1.2 Notation of scheduling models

Before we begin the actual discussion of scheduling algorithms, we introduce a universal notation for a scheduling problem: $\alpha|\beta|\gamma$, where α represents the machine environment, β the boundary conditions and γ function we wish to minimize. When an algorithm is optimal for a particular scheduling model, then we will be able to compactly notate the exact model for which the optimality holds in this way.

In this section, $\alpha = 1$, which indicates that the machine has exactly one processor. If we note $\alpha = Pm$, then there are m identical processors present. Thus, any task can be performed on any of these m processors and the running time is the same on any machine. The possibilities for β that are of interest in this course are: r_j , *pmtn* and *prec*. r_j indicates that the tasks have arrival times that are different from zero. In this case, r_j indicates the arrival time of task j. A task can never be started before it has arrived. *pmtn* indicates whether tasks may be interrupted, while *prec* indicates that some tasks can only be executed if certain other tasks are executed first. If, we do not specify r_j , *pmtn* or *prec* then they are assumed not to apply.

Denote C_j as the completion time of task j (under a given schema discipline). In our discussion of

non-real-time algorithms, for γ, the function we wish to minimize, there are two possibilities: $\gamma = \Sigma C_j$ or $\gamma = \Sigma w_j C_j$. In the first case we wish to have the sum of the completion times as small as possible, in the second case the weighted sum.

For example, $1||\ \Sigma C_j$ indicates that the tasks are executed on a machine with one processor, that we want to minimize the sum of the completion times, and that the tasks can all be executed from time zero. Furthermore, no task interruption is allowed and there are no tasks that must be executed before some other tasks. If we do interrupt a task, we note this as $1|pmtn|\ \Sigma C_j$, etc.

1.3 Scheduling algorithms

We start with some of the best known and most studied scheduling algorithms. During this overview we also distinguish between systems in which a process/task can be interrupted, called *preemptive* systems, and those in which a task is performed continuously (as in some production systems), or non-preemptive systems.

Proces	Arrival time	Service time (T_s)	Start time	Completion time	Turnaround time (T_q)	T_q/T_s
A	0	1	0	1	1	1
B	1	100	1	101	100	1
C	2	1	101	102	100	100
D	3	100	102	202	199	1.99
Mean					100	26

Fig. 38: FIFO discipline example [Fink 88].

First-Come-First-Served (FCFS): The simplest discipline is to serve the processes from the ready queue in the order in which they were placed in the queue. The process that has been ready the longest is selected first.

The major drawback of this approach is that small tasks, which can be executed very quickly, often end up behind large processes that use the processor for a long time (see Figure 38). In other words, the average turnaround time is often far from optimal, while the normalized turnaround time is even worse.

If we also take into account that we can divide processes into processor- and I/O-bound processes, we notice that I/O-bound processes are strongly disadvantaged. This is because every time an I/O operation is performed, they have to sit at the back of the ready queue again.

FCFS, also called First-In-First-Out (FIFO), is mainly interesting in combination with priority mechanisms. A topic we will come back to further.

Round Robin (RR): One way to counteract the dominance of long-running tasks in a FIFO system is to set a maximum amount of time that a single process can have uninterrupted access to the processor. After this time, which is often referred to as *quantum*, the task in progress is interrupted and the process must again take its place at the end of the ready queue. When we use an RR discipline with a quantum of 5 in Figure 38, we see that process *C* suddenly realises a much lower waiting time.

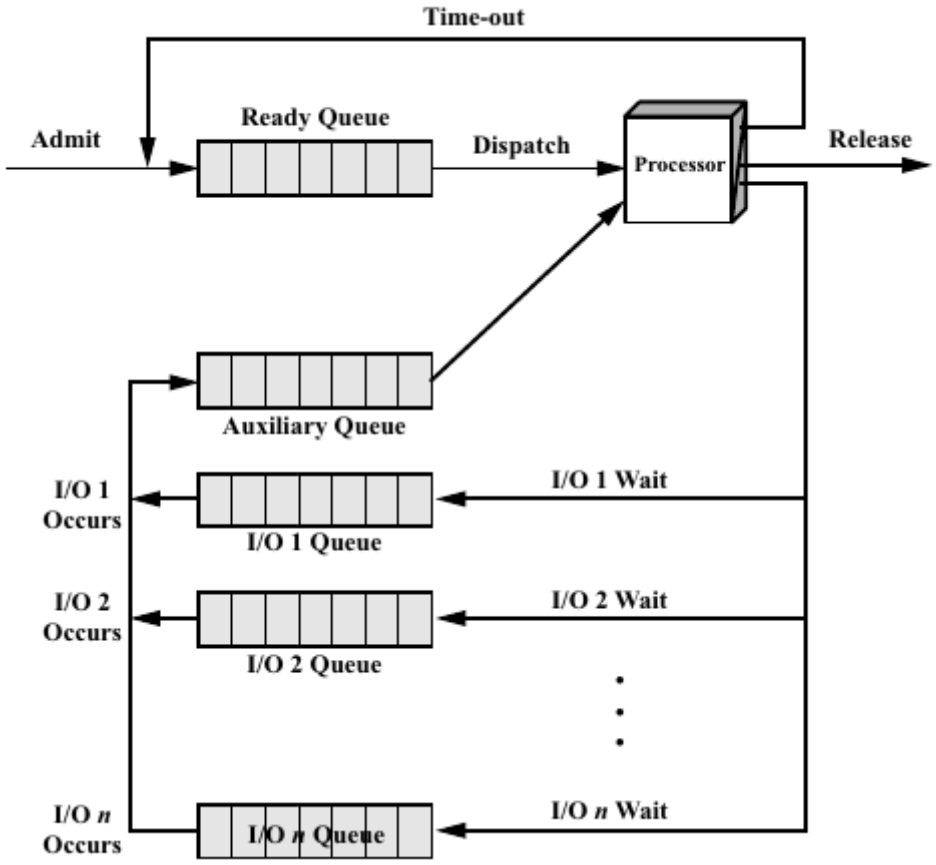

Fig. 39: Fairness between I/O and processor-bound processes at RR [Stallings03].

An important choice in the RR discipline is the length of the quantum *q*. A very short quantum will, in theory, always give rise to a lower average execution time. However, in practice one must also take into account the time needed to switch two processes. A too small value *q* for the quantum, implies that the processor is more occupied with switching processes, than with executing them. The limit case where the quantum *q* becomes infinitely small is known as *processor sharing*. This model is not practically realizable, but an evaluation of it can have important implications for systems with relatively small quanta and is often mathematically less complex.

Although RR partially solves the problem of large normalized turnaround times, it still puts I/O-bound processes at a disadvantage. Indeed, often an I/O operation will already take place before the quantum of the process is over. Processor-bound processes, for their part, do use their quantum fully each time. A possible approach to counter this unfairness is shown in Figure 39. Processes that become ready again after waiting for an I/O event, take place in a separate queue. When this queue contains processes, they are served first. The time they are allowed to have access to the processor is equal to the quantum *q* minus the time they have had access to the processor since the last time they were selected from the ready queue. Some bookkeeping is required to implement this scheme.

Shortest Process Next (SPN): With this discipline, the process that requires the least processor time (that is, the shortest process) is always chosen from the ready queue. When we need to schedule a number of tasks, all present at time 0, we can easily show that this order gives rise to the lowest possible average turnaround time (assuming the process can be executed in one go). In short, using our notation, we state that SPN is optimal for

$$1||\sum C_j..$$

We can easily prove this (for non-preemptive systems, the result for preemptive systems follows from the SRTN result). Suppose that the processing time of task *j* is equal to p_j , for $j = 1, \ldots, N$.

Suppose further that we first perform task 1, then task 2, and so on. Then the total turnaround time (T_q) will be equal to

$$T_q = p_1 + (p_1 + p_2) + \ldots + (p_1 + \ldots + p_N) = \sum_{i=1}^{N} (N - i + 1) p_i. \qquad (3)$$

When we arrange the tasks such that $p_1 \leq p_2 \leq \ldots \leq p_N$, then this expression is minimal. In short, if we cannot interrupt tasks and all tasks are present at time 0 then SPN is optimal for the average waiting time.

However, when all tasks are not yet in the system at time zero, then this is no longer the case, or yet SPN is not optimal for

$$1|r_j|\sum C_j,$$

where we note r_j as the arrival time of task j. We see this by means of the following simple example. Suppose we have two tasks with $p_1 = 100$, $p_2 = 1$, $r_1 = 0$ and $r_2 = 1$. SPN results in an average turnaround time of (100 + 100)/2 = 100. However, if the processor remains empty until time 1 and then first performs task 2, the average turnaround time is equal to (102 + 1)/2 = 51.5. We will discuss this problem in more detail when discussing the SRTN discipline.

Although SPN is optimal for scheduling a batch of tasks, this discipline is difficult to implement, as it requires prior knowledge of the running time of all processes. In a production environment it sometimes happens that a set of the same tasks is executed many times. In that case, the processing time of the tasks can be estimated and used for scheduling future tasks. Note that this technique is disadvantageous for large tasks.

Weighted Shortest Process Next (WSPN): This algorithm, also known as Smith's rule, does not process the tasks according to increasing length p_j, but according to p_j / w_j where $w_j \geq 0$ is a weight. Suppose all tasks are present at time 0. Recall that we have noted the completion time of task j as C_j. Then WSPN will minimize the average weighted completion time, or WSPN is optimal for

$$1||\sum_{j=1}^{N} w_j C_j.$$

We prove this for a moment for non-preemptive systems.
Assume that another order S' is better. Number the tasks so that they are completed in order. Then there exists an i with $p_{i+1}/w_{i+1} < p_i / w_i$, i.e. $p_{i+1} w_i < p_i w_{i+1}$. We can rewrite $\sum_{j=1}^{N} w_j C_j$ as

$$w_1 p_1 + w_2(p_1 + p_2) + \ldots + w_N(p_1 + \ldots + p_N) = \sum_{k=1}^{N} p_k \left(\sum_{l=k}^{N} w_l \right). \qquad (4)$$

If we swap task i and i + 1 then only the i-th and i + 1-th term of

$$p_i(w_i + \ldots + w_N) + p_{i+1}(w_{i+1} + \ldots + w_N)$$

to

$$p_{i+1}(w_i + \ldots + w_N) + p_i(w_i + w_{i+2} + \ldots + w_N)$$

The difference between the two is $p_i w_{i+1} - p_{i+1} w_i > 0$, which indicates that the weighted completion time has decreased. This is in contradiction with the minimality of S'.

Shortest Remaining Time Next (SRTN): We can refine the SPN algorithm when tasks can be interrupted (we assume that the interruption itself does not waste any time). SRTN will always check when a new process becomes ready whether this new process needs less running time than the remaining processing time of the current process. If this is the case, the current process will be interrupted and replaced by the new process. In this way the process that needs the least processing time will always be present in the processor.
SRTN is optimal when it comes to minimizing turnaround time. Regardless of whether the tasks are already all present from time 0 (unlike SPN) and when we allow interruption:

$$1|r_j, pmtn| \sum C_j,$$

since the average turnaround time is then equal to $\sum_{j=1}^{N}(C_j - r_j)/N$ and r_j has a fixed value, it suffices to minimize $\Sigma\, C_j$.

Suppose that another discipline *S'* is better. Then there exists a time *t* when task *j* is executed while another task *i* is present (i.e., $t \geq r_i$) that has a smaller remaining running time. We note the remaining duration as p^r_j and p^r_i , respectively. Next, we swap task *i* and *j* so that during the time that task *i* or *j* was active, we first fully execute *i* during the p^r_i units. Then we let task *j* execute in the remaining p^r_j time. If we note C_j' and C_i' as the new completion times of both tasks then $C_j' = max\{C_i , C_j\}$ and $C_i' < min\{C_i , C_j\}$. Consequently, we see that

$$\sum_k C_k = \sum_{k \neq i,j} C_k + C_i + C_j > \sum_{k \neq i,j} C_k + C_i' + C_j'. \qquad (5)$$

Since the completion times of the other tasks do not change, $\sum_k C_k$ was not minimal and we obtain a contradiction on the optimality of *S'*.
Although SRTN is optimal, it requires perfect knowledge of the length of all tasks and also the remaining processing time of all tasks must be tracked. In addition, starvation effects occur for the longer tasks, as they are continuously passed by short tasks.

Nonpreemptive SRTN (nSRTN): We can also use the result of the SRTN algorithm to create a new scheduling discipline that tries to minimize the running time when tasks are not interrupted where all tasks do not have to be present at time 0. Thus, the following model is involved:

$$1|r_j| \sum C_j.$$

It has been shown that finding the optimal solution is NP-hard. However, using the SRTN result, we can construct a sequence such that the sum of the completion times is at most twice as large as the optimal one, such an algorithm is called a 2-*approximation.*

We proceed as follows. Suppose we renumber the tasks so that the SRTN discipline completes them in order: $C_1 \leq C_2 \leq \ldots \leq C_N$. Then we construct a new discipline with completion times $\bar{C}_1 \leq \bar{C}_2 \leq \ldots \leq \bar{C}_N$, by completing the tasks in the same order, possibly leaving the processor empty for a while until the task in question arrives (think back to the SPN example with the two processes). We call this discipline nonpreemptive SRTN.

This new discipline will give rise to an average turnaround time that is at most twice as large as the

optimal one. To prove this, it suffices to show that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$. For the best algorithm that may not use interruptions can never outperform the optimal algorithm that may use interruptions.

If we look at the completion time of the j-th task $\bar{C}_j$, it is maximal if the first task is the last of the first *j* tasks to arrive (because an idle period always ends with the arrival of a new task):

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^{j} p_k. \qquad (6)$$

Furthermore, $r_k \leq C_k \leq C_j$ for $k \leq j$, consequently $\max_{k \leq j} r_j \leq C_j$. Further, since the tasks are completed in order, $\sum_{k=1}^{j} p_k \leq C_j$. It follows that $\bar{C}_j \leq 2 C_j$, which is sufficient to prove the statement.

We cannot tighten this upper bound any more, because there are examples where the average running time is up to 2 times larger. For example, consider 2 tasks with $p_1 = N$, $p_2 = 1$, $r_1 = 0$ and $r_2 = N - 2$. Then the average running time will be equal to $(N + 3)/2$ in the case of SPN. SRTN does slightly better with $(N + 1 + 1)/2$. However, for nonpreemptive SRTN, the processor remains empty during the first $N - 2$ units and consequently the average running time is equal to $(2N - 1 + 1)/2 = N$. The limit for N to infinity of $2N/(N + 2)$ is equal to 2. An overview of these theoretical results can be found in Figure 40.

Algoritme	Eigenschap	Systeem
SPN	optimaal	$1 \mid\mid \sum C_j$
WSPN	optimaal	$1 \mid\mid \sum w_j c_j$
SRTN	optimaal	$1 \mid r_j, pmtn \mid \sum C_j$
nSRTN	2-approximatie	$1 \mid r_j \mid \sum C_j$

Fig. 40: Theoretical Results for Uniprocessor Scheduling

Highest Response Ratio Next (HRRN): This discipline uses the weighted turnaround time (T_q / p_s) when selecting a process. Specifically, each process has a preliminary weighted turnaround time equal to $RR = (w + p_s)/p_s$, with w equal to the time the process was already waiting for the processor. At startup, $RR = 1$, then the value increases with the wait time. The increase is proportional to the processing time p_s of the process. Therefore the *RR* of shorter processes will increase faster.

The HRRN discipline, when a process releases the processor, will select the process with the largest *RR* value. We notice that short processes get an advantage, because their *RR* increases faster. On the other hand, the starvation effect of the longer processes is countered by the fact that they eventually obtain the largest *RR* value. This discipline requires, just like all the previous ones, knowledge of the processing time of the processes (or at least an estimate of this).

Multilevel Feedback Scheduling: Feedback scheduling tries to mimic the behavior of the previous schemes, without having the (estimated) processing times. The idea is not to look at the time that a process needs, information that we do not have, but to look at the time that a process already had at its disposal. As this time increases, we want to allow a process to access the processor less frequently.

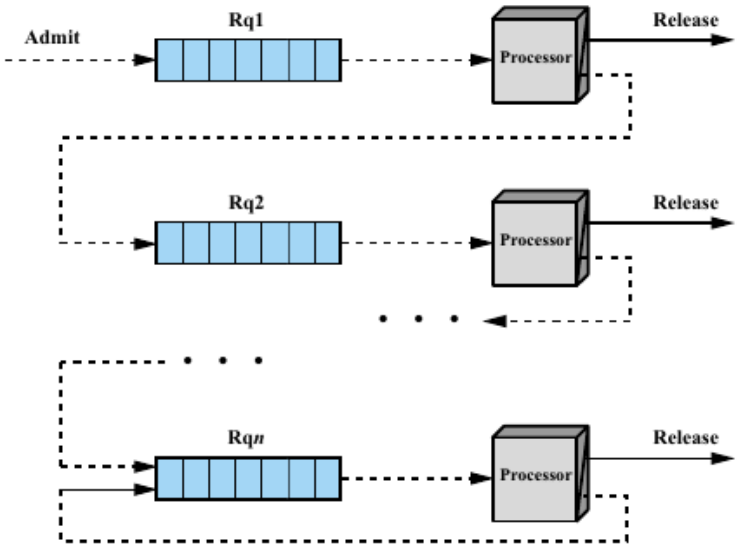

Fig. 41: Multilevel Feedback Scheduling [Stallings 03].

A possible implementation consists of handling *n* ready queues, which are controlled with a priority mechanism (see Figure 41). This means that processes in the i-th queue can only dispose of the processor if the *i - 1* previous queues are empty. A process that becomes ready will take its place in the first queue. When the processor selects a process from queue i, it has access to the processor for one quantum. It then takes its place in the back of the *i + 1* queue (unless *i = n*, in which case it takes its place in queue *n* again). When a process is finished, it disappears from the system. Each of the queues is served in an FCFS manner, except the last one which uses an *RR* discipline.

We note that short tasks are favored in this way, as they descend less far down the queue. To prevent starvation of long processes, the quantum q will often increase with the index *i* of the queue, for example $q = 2^{i-1}$. Furthermore, I/O-bound processes have the advantage that they always take their place in queue 1, after they have performed an I/O operation. Finally, Figure 42 gives an overview of the various disciplines by way of an example.

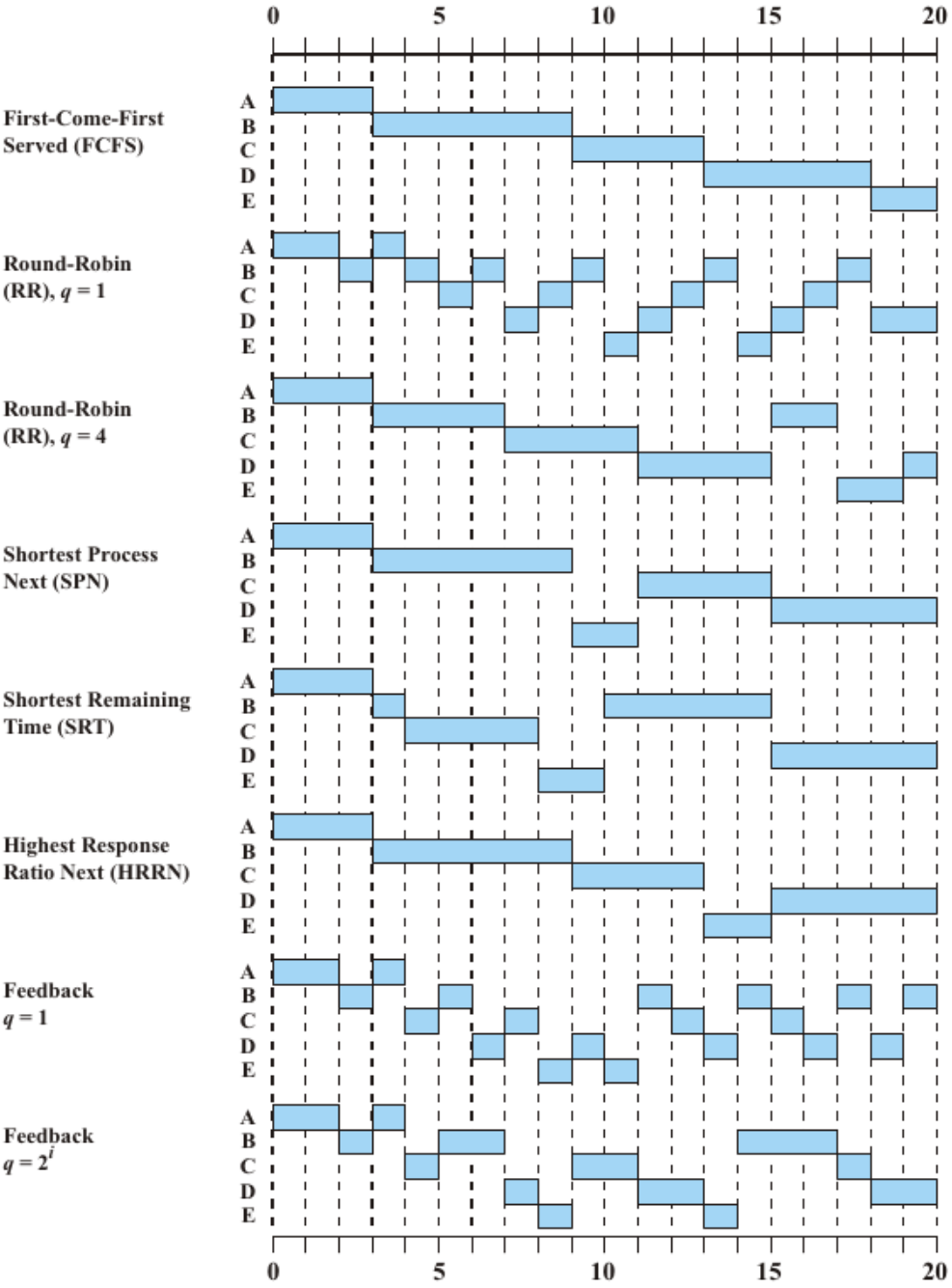

Fig. 42: Scheduling example with 5 processes with $r_i = 2i - 2$ [Stallings 03].

1.4 Traditional UNIX scheduler

In this section we will take a closer look at how a traditional UNIX scheduler works. This scheduler works in a similar way to a multilevel feedback queue. The scheduler divides the time into intervals of equal length (typically 1 second) and each process j has a priority $P_j(i)$ during interval i. The lower this value, the higher the priority. Furthermore, the scheduler also has a *scheduler resolution*, which indicates how often a clock interrupt is generated per second, typically 60 interrupts per second.

For each process j a counter $U_j(i)$ is kept which indicates how often the process was active during the current interval i, with each interrupt the counter of the active process will be increased by one. Using this counter, at the end of each interval, an exponentially weighted average of the CPU usage is calculated, as follows

$$CPU_j(i) = U_j(i)/2 + CPU_j(i - 1)/2.$$

Next, the priority of all processes is adjusted using the formula below:

$$P_j\,(i) = Base_j + CPU_j\,(i - 1)/2 + nice_j\,,$$

where the use of $Base_j$ and $nice_j$ will be explained later. With each clock interrupt, the process with the highest priority, i.e., the lowest $P_j\,(i)$ value, acquires the CPU. Usually there are multiple processes with the same priority and round-robin is used. Thus, as with the multilevel feedback queue, we see that the priority of a process that could dispose of the CPU will decrease. However, unlike the classical multilevel feedback queue, the priority of inactive processes will increase automatically because the variable $CPU_j\,(i)$ will be smaller than $CPU_j\,(i - 1)$. In this way, starvation of large processes will be automatically prevented.

Note also that new processes may emerge all the time during an interval i, which may have a higher priority than the process currently running. In this case, the process in progress will have to give up the CPU to the new process at the next clock interrupt.

The $Base_j$ parameter makes sure the processes are divided into different classes. For example, swap processes and some I/O support processes will always have a higher priority than user processes. By making the $Base_j$ of the user processes sufficiently large, the swap process will always have a smaller $P_j\,(i)$ value, even when it was very often active during the last intervals.

Time	Process A		Process B		Process C	
	Priority	CPU Count	Priority	CPU Count	Priority	CPU Count
0	60	0 1 2 • • 60	60	0	60	0
1	75	30	60	0 1 2 • • 60	60	0
2	67	15	75	30	60	0 1 2 • • 60
3	63	7 8 9 • • 67	67	15	75	30
4	76	33	63	7 8 9 • • 67	67	15
5	68	16	76	33	63	7

Fig. 43: Example of a traditional UNIX scheduler [Stallings 03].

The $nice_j$ parameter can be set by the user. Default it is equal to zero. However, the user can increase it to 19 in order to prioritize other processes more often.

1.5 Fair-share scheduler

Although many scheduling disciplines provide fairness between different processes, there is often a demand for fairness between groups of processes as well. For example, one could group all

processes per user and realize fairness between users. Or if multiple organizations share a single (expensive) machine, then it is desirable to divide the resources between these organizations in proportion to the investments made.

A possible solution in the case of classic UNIX schedulers consists of adding another counter $GU_k\ (i)$ per group *k*. It will increase by one for each clock interrupt, if a process from group k was active. This counter is used analogously to the variable $U_j\ (i)$ to adjust the variable $GCP\ U_k\ (i)$ at the end of each interval:

$$GCP\ U_k\ (i) = GU_k\ (i)/2 + GCP\ U_k\ (i - 1)/2.$$

This variable will in turn influence the priority of a process *j* from group *k* as follows:

$$P_j\ (i) = Base_j + CPU_j\ (i - 1)/2 + GCP\ U_k\ (i - 1)/4W_k + nice_j\ ,$$

where W_k is a group weight, with $\sum_k W_k = 1$, indicating the percentage of CPU allocated to group *k*. A double value for W_k should result in twice the frequent use of CPU. If there are many processes in a single group, then the term $GCPU_k\ (i-1)/4W_k$ will often be decisive in determining the priority $P_j\ (i)$. This term will determine which group(s) can dispose of the CPU. The term $CPU_j\ (i - 1)/2$ indicates which of the processes belonging to this group can effectively dispose of the CPU.

2 Real-time short-term scheduling

In this section, we are introduced to some real-time scheduling algorithms. The main difference with their non-real-time counterparts, is that with a task *i*, there is also a deadline d_i associated. This deadline indicates when the task must be completed at the latest. In some cases, it is also possible that a deadline rests on the start time instead of the completion time, this is mainly the case for non-preemptive systems.

Real-time scheduling solutions are often divided into static and dynamic solutions. Static solutions are executed off-line, i.e. these algorithms are executed before the system is operational. We distinguish two types:

1. Static *table-driven* solutions: In this case, a table is created in advance that perfectly represents which tasks will be executed when. If not all tasks can meet their deadline, some tasks will not be executed.

2. Static *priority-driven* solutions: In contrast to table-driven solutions, no table is produced as output, but each task is given a priority as a result of the scheduling algorithm. A classical priority-based scheduler takes care of the actual execution of the tasks.

When we opt for a dynamic solution, decisions will be taken on-line. In other words, while the system is operational, decisions are taken regarding the acceptance of new tasks. This can be done in a *best-effort* manner, where all tasks are initially accepted and those that do not meet their deadline are stopped. It is also possible that a task is only accepted if there is certainty that it can be executed within the set deadline. This has the advantage that no CPU time is lost on the partial execution of tasks. The disadvantage is of course the complexity that comes with such an implementation.

It is also important to note that not all tasks have to have deadlines in a real-time system. Furthermore, some deadlines may be *hard*, meaning that they should never be crossed, or *soft*, meaning that a small crossing is still solvable. Unlike a classic UNIX scheduler, some real-time schedulers will also be able to interrupt a lower priority process faster than the next clock interrupt. This can be important since clock interrupts only occur every few milliseconds, while for example

military flight simulators wish to support interrupts that take effect within 100 microseconds.

We will discuss two groups of static scheduling solutions: deadline scheduling, which provides a table-driven solution, and rate monotonic scheduling, which is an approach that belongs to the group of priority-driven solutions.

2.1 Deadline scheduling

We assume that we know all the tasks to be executed, know their duration p_i and deadline d_i. In a real-time setting, the assumption that we know the execution time is more realistic, after all it also affects the deadline. As before, we note C_j as the completion time of task j. We set $U_j = 1$ if $C_j > d_j$, indicating that task j has not been completed on time, otherwise we set $U_j = 0$. We are now not so much interested in the average turnaround time $1/n \sum_{j=1}^{n} C_j$, but in the number of tasks $\sum_{j=1}^{n} U_j$ that fail to meet its deadline or in the maximum degree to which a task is late, being $L_{max} = \max_{j=1}^{n}(C_j - d_j)$. Thus, in terms of our scheduling model notation, we have two additional possibilities for γ, the function we wish to minimize. Note, if $\gamma = L_{max}$ or ΣU_j , then this implicitly indicates that the tasks have a deadline, thus we do not need to specify d_j as a boundary condition in β .

Earliest Deadline First (EDF) - Earliest Due Date (EDD): This discipline, also known as Jackson's rule, tries to minimize the maximum degree to which a task is late, or rather, to make L_{max} as small as possible. It states that we execute the tasks in order of increasing deadline d_i and this without interruption, i.e., we order the n tasks such that $d_1 \leq d_2 \leq \ldots \leq d_n$. Note, no knowledge of the running time p_i is required. We will prove that this simple discipline minimizes $\max_{j=1}^{n}(C_j - d_j)$, if all tasks can be performed from time 0 (that is, the arrival times r_i are equal to zero). The following system is involved:

$$1||L_{max}.$$

Suppose S is an other order. Then there exist two tasks j and k that are performed in close succession with $d_k < d_j$, otherwise S is the EDF order. By switching them in order we only change the terms $C_j - d_j$ and $C_k - d_k$ to $\max_{i=1}^{n}(C_i - d_i)$ and obtain a new discipline S' . Suppose that task j started at time t in case of order S, then $C_j = t + p_j$ and $C_k = t + p_j + p_k$. By interchanging, we see that $C'_j = t + p_k + p_j = C_k$ and $C'_k = t + p_k$. Consequently,

$$C'_j - d_j = C_k - d_j < C_k - d_k ,$$

and

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k .$$

In short, $\max_{i=1}^{n}(C'_i - d_i) \leq \max_{i=1}^{n}(C_i - d_i)$ and the order S' is more like the EDF order than S. Repeating this argument, we see that this maximum for the EDF order is less than or equal to the maximum for the order S, which was chosen arbitrarily.

A very important consequence of this result is, that a set of tasks with associated deadlines can only be realized if also the EDF order meets all deadlines, or even, that $L_{max} = \max_{j=1}^{n}(C_j - d_j) \leq 0$ for EDF. In fact, we can easily[1] prove that EDF meets all deadlines if and only if

$$u_i = \frac{1}{d_i} \sum_{k:d_k \leq d_i} p_k \leq 1.$$

1 That this condition is necessary is obvious. The converse follows immediately by stating that if EDF cannot schedule the tasks, at least one of these conditions must fail if for every deadline d_i applies

$u = max_i\ u_i$ is called the loading factor or load. In short, if the load is less than 1 then EDF will be able to schedule the tasks without missing any deadline. However, EDF does not have a good performance when it comes to the number of tasks that do not meet their deadline when $u > 1$. An example of EDF scheduling is shown in Figure 44.

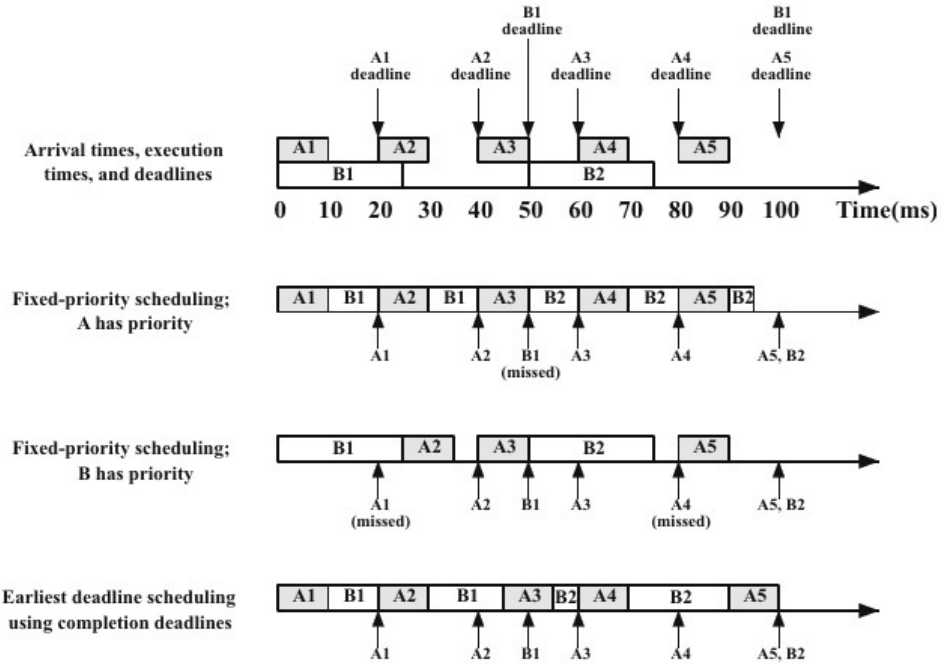

Fig. 44: Example of EDF scheduling versus priority scheduling [Stallings 03].

Moore-Hodgson Algorithm (MHA): This algorithm looks at the same problem as the EDF algorithm, but tries to minimize the number of tasks that fail to meet its deadline $\sum_{j=1}^{n} U_j$. It works as follows:

S1: This algorithm first sorts all tasks in EDF order such that $d_i \leq d_{i+1}$ for $i = 1, \ldots, n - 1$. As a result, we obtain a set of n tasks.

S2: Next, we check whether there is (still) a task in the sequence that would be executed too late. If this does not exist, then the order is optimal, otherwise we continue with Step S3.

S3: Let J_α be the first task in the sequence that is overdue. Then take a job J_β with $p_\beta = \max_{i=1}^{\alpha} p_i$, or another task as long as possible from the first α tasks in the sequence and remove it from the sequence. Adjust the completion times of the tasks that were executed after J_β return to Step S2.

One can prove that this algorithm removes the smallest possible number of tasks so that the deadlines of all remaining tasks are respected. So it is optimal for following system:

$$1||\sum U_j$$

While this is easy to prove, let's leave the proof for what it is.

Arrival times and preemptions: If we impose the additional requirement that each task also has an arrival time r_i and we do not allow preemption, however, both problems discussed are NP-hard.

Solutions in polynomial time are available if we combine arrival times with preemptions. For example, EDF is optimal for

$$1|r_j, pmtn|L_{max}$$

and will always meet all deadlines if the load is less than or equal to one. In the case where there are arrival times $r_j > 0$, the load is defined as $u = \sup_{t_1<t_2\leq\infty} u_{[t_1,t2)}$ with

$$u_{[t_1,t2)} = \frac{1}{t_2 - t_1} \sum_{j:t_1\leq r_j,d_j\leq t_2} p_j.$$

Even in the non-preemptive case, EDF is still optimal if we restrict ourselves to *no-idling* scheduling orders. These do not allow the processor to be idle for a while, while still allowing a task to be started. The proof is analogous to the previous one by noting that for two orders that both do not allow idling, the processor will be idle at the same times.

Precedence conditions: In practice, when we wish to perform a set of real-time tasks, it may happen that we cannot just execute all the tasks in any order. The execution of task *i* can sometimes only be done after another task *j* has been executed. We then say that a *precedence* condition or relationship exists between task *i* and *j*. We can characterize all these conditions using a directed graph $G = (V,E)$ by representing all tasks as nodes in V and the precedence relations as edges in E. This graph must not contain any closed paths, otherwise every possible sequence will break one of the precedence rules. In short, the graph must be acyclic.

If we assume that we know the duration p_j and the deadline d_j of all tasks and that the arrival times r_j are equal to zero, then a Lawler's algorithm will minimize the maximum extent to which the deadlines are missed $L_{max} = max_j (C_j - d_j)$, it is thus optimal for

$$1 \mid prec \mid L_{max}$$

and it works like this:

S1: Let $num = n$ be the total number of tasks and G be the graph with the precedence conditions. Further, suppose that $U(G)$ contains the nodes in G from which no edges depart. Assume that T is equal to the execution time of all tasks together, i.e., $T = \sum_{j\in V} p_j$.

S2: Choose a task j in $U(G)$ for which $T - d_j$ is minimal (or still, for which d_j is maximal). This task is given number *num*. Decrease *num* by one, set $T = T - p_j$ and remove the node j from the graph G. Redefine the set $U(G)$ and repeat step S2 until $num = 0$.

The tasks are executed in the order indicated by the *num* value obtained during step S2. In short, the order of execution is recorded in reverse order. A graphical representation of the operation of Lawler's algorithm is shown in Figure 45.

When we also bring arrival times $r_j > 0$ into play and we do not allow preemptions then we are again dealing with an NP-hard problem (since no precedence is a special case of precedence with G= (V, E) such that E is empty). If we do support preemption then there again exists an optimal solution that can be found in polynomial time (being with the help of Baker's algorithm).

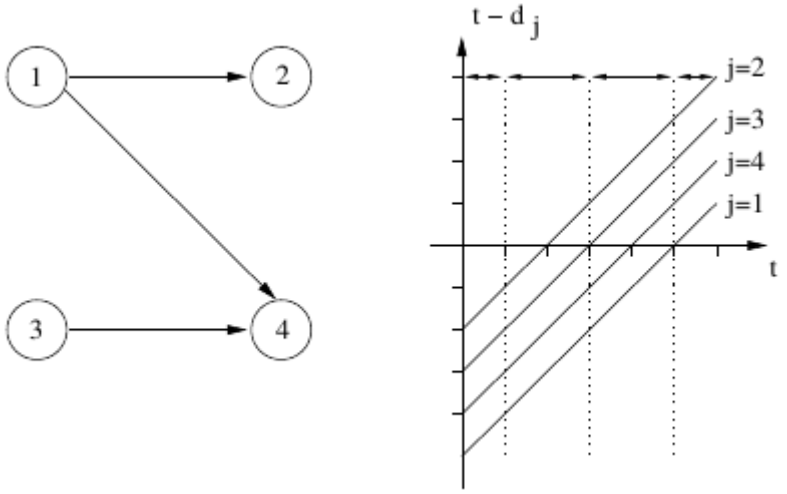

Fig. 45: Example of precedence scheduling using Lawler's algorithm: $p_1 = 1$, $p_2 = 2$, $p_3 = 2$, $p_4 = 1$ and $d_1 = 5$, $d_2 = 2$, $d_3 = 3$, $d_4 = 4$. The optimal order is 1, 2, 3, 4 and $L_{max} = 2$.

2.2 Rate monotonic scheduling (RMS)

In this section, we focus on the problem of scheduling periodic tasks. We suppose that we have *n* periodic tasks. All instances of task *i* have a duration p_i and the period of the task *i* instances is equal to T_i , which implies that we wish to execute the task once every T_i time units. In other words, for each task, an infinite stream of instances is generated with arrival times $r_j = jT_i$, for $j \geq 0$. As the *relative* deadline for task *i*, we take T_i, which implies that an instance of task *i* must be executed before a new instance of the same task arises.

The percentage of time that a task loads the processor, or still, the utilization, is clearly equal to p_i / T_i. Before discussing rate monotonic scheduling, we show that a set of *n* periodic tasks can be realized with EDF if and only if

$$U = \sum_{j=1}^{n} p_j/T_j \leq 1.$$

We first show that the load $u \leq U$. Namely, for all $t_1 < t_2$ we have

$$u_{[t_1,t2)}(t_2 - t_1) = \sum_{j:t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^{n} \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j \leq (t_2 - t_1)U,$$

because there are at most $\left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor$ instances of task *j* that arrive in the interval *[t1 , t2)* and that also have a deadline smaller than t_2. Conversely, we need to show that $u \geq U$, from which it then follows that $u = U$. We do this by assigning $t_1 = 0$ and t_2 to the smallest common multiple *H* of T_1 , . , T_n. Then

$$u_{[0,H)}H = \sum_{j:0 \leq r_j, d_j \leq H} p_j = \sum_{j=1}^{n} \frac{H}{T_j} p_j = HU.$$

Since *u* is the supremum over all intervals, $u \geq U$. Herewith the result is proved, since EDF can schedule everything without a deadline crossing if and only if the load $u = U \leq 1$.

Rate Monotonic Scheduling (RMS): While EDF is optimal for scheduling periodic real-time

tasks, in the sense that it can utilize the maximum capacity of the processor, it is often the case that not all tasks are real-time or that not all tasks have hard deadlines. An interesting alternative can be offered in this case by assigning a priority to each instance of task *i* and integrating it into a rather classical UNIX scheduler. In this case, the priorities of the real-time tasks will be higher than those of all other processes and will not be changed during execution.

RMS, developed by Liu and Layland, states that we order the tasks such that $T_1 \leq T_2 \leq . \ldots . \leq T_n$ and give the instances of task *i* the i-th highest priority. One can then show that RMS will always respect the deadlines if

$$U = \sum_{j=1}^{n} p_j/T_j \leq n(2^{1/n} - 1).$$

For $n = 1$, this value is equal to 1, as n increases, it decreases to $ln\ 2$ (we see this via L'Hopitals rule since the derivative of $(2^{1/n} -1)$ is equal to $-2^{1/n}\ ln\ 2/n^2$). However, this is only a sufficient condition and not a necessary condition. RMS is optimal within the class of static priority-driven solutions (if the relative deadlines are equal to the period). Furthermore, RMS will also always respect all deadlines if all periods are multiples of each other and $U \leq 1$